

Understanding AMIRIS

A Complete, Plain-Language Guide

What AMIRIS is, how it works, what every file and folder means, what 'backtesting' means, and a full walkthrough of the work done so far - written so that no prior background is required to follow along.

Prepared 2026-07-14

1. The Big Picture: What Problem Are We Solving?

Every hour of every day, Germany's electricity has a price. This price is set by an actual marketplace - power plants offer to sell electricity, and buyers (utilities, industry, etc.) bid to buy it, one hour at a time, for the next day. This is called the 'day-ahead market'. The result is 8,760 individual prices every year (24 hours x 365 days) - one price per hour, in Euros per Megawatt-hour (EUR/MWh).

A university department needs exactly this kind of hourly price data as an input to its own regional energy planning model (for the Thuringia region of Germany). Right now, the department pays a private company, Energy Brainpool, to produce and sell this data to them.

The thesis this work supports asks a simple question with a not-so-simple answer: could a free, open-source computer program called AMIRIS produce electricity price data that is just as trustworthy as the data currently being purchased? If so, the department could stop paying for it. If not, we need to understand exactly where and why AMIRIS falls short.

In one sentence:

AMIRIS is a computer simulation of the German electricity market; this project is testing whether it can be trusted as a free substitute for a commercial data product.

2. What Exactly Is AMIRIS?

AMIRIS stands for 'Agent-based Market model for the Investigation of Renewable and Integrated energy Systems'. It is built and maintained by the German Aerospace Center (DLR), a major German research institution. It is free, open-source, and has been used in real published energy research for years.

The 'agent-based' idea, explained simply

Instead of using one giant mathematical formula to guess a price, AMIRIS creates many small, separate computer characters - called 'agents' - each one standing in for a real type of participant in the electricity market. For example, one agent represents all of Germany's nuclear power plants; another represents all its wind turbines; another represents the electricity buyers; another represents the market itself, which matches buyers and sellers.

Each agent has its own simple rules of behaviour (for instance: 'offer your electricity for sale at a price close to what it costs you to produce it'). Every simulated hour, all the agents place their offers and bids into the simulated market, the market matches them up (cheapest sellers first), and wherever supply exactly meets demand, that becomes the price for that hour. This is repeated 8,760 times to produce a full simulated year.

Analogy:

Think of a very detailed game of make-believe: instead of one formula predicting the weather, you

create thousands of tiny simulated raindrops, winds and temperatures, let them interact according to physical rules, and watch what emerges. AMIRIS does the same thing for an electricity market: many small, rule-following virtual companies, and a real price emerges from their interactions.

The engine and the toolbox

- The actual simulation is run by a program written in Java (a well-known programming language) - this is the 'engine' that does the heavy computational work.
- Around that engine sits a set of Python tools (amirispypy and fameio) that prepare the inputs, launch the engine, and turn its raw output into readable spreadsheets (CSV files).
- A full simulated year currently takes well under a minute to run on an ordinary laptop.

3. What Does 'Backtesting' Mean?

Backtesting means: run the simulation for a year that has already happened in real life, feeding it the REAL conditions from that year (how much electricity people actually used each hour, how windy or sunny it actually was, what coal and gas actually cost) - and then check whether the price the simulation calculates matches the price that actually occurred.

Analogy:

Imagine testing a weather-forecasting computer program by feeding it the actual, known temperature and wind readings from a storm that happened last year, and checking whether it would have correctly worked out what the weather did next. If it reproduces the known outcome well, you gain confidence that its internal logic is sound - not that it can predict the future, but that its reasoning process is realistic.

This is an important distinction: AMIRIS is not asked to guess unknown things like the weather or fuel prices - those are handed to it as known facts (because the year already happened). Its only job is to work out how a realistic market, full of realistic buyers and sellers, would have turned those known facts into an hourly price. Backtesting checks whether that price-setting logic is trustworthy.

4. The AMIRIS Folder Structure, Explained

On the computer, the AMIRIS project lives in one main folder. Inside it are a few purpose-built areas:

Folder / File	What it is, in plain terms
amiris-core...jar	The simulation 'engine' itself (a runnable Java program)
py313env\	The Python toolbox - all the helper software needed to prepare inputs and read outputs
examples\backtest\	A library of ready-made 'recipes', one per historical year, for reproducing that year's market
result\ (and result_Germany<year>\)	Where the answers land after a simulation finishes running

Folder / File	What it is, in plain terms
Data\, price_comparison_*.csv/png	Real historical prices we downloaded, plus our own comparison results

Inside a 'recipe' folder (e.g. Germany2018)

Each historical year that AMIRIS can reproduce has its own folder under examples\backtest\ (for example Germany2015, Germany2016, ... Germany2019). Each one is a complete, self-contained 'recipe' for rebuilding that year's electricity market from scratch. Inside, you'll always find:

File / Folder	Plain-language meaning
scenario.yaml	The master recipe card: which year, when it starts/ends, and which other files to include
schema.yaml	A technical rulebook the software uses internally (not something a person edits by hand)
agents\	Description cards for every 'virtual company' in the simulation - how big is the nuclear fleet, how much wind and solar capacity exists, how big is the battery storage, etc.
contracts\	The rulebook for who is allowed to trade with whom (e.g. power plants sell through traders, traders sell into the market)
timeseries\	The real historical 'ingredients' for that year - see next section
README.md	A short human-readable description of that year's scenario and any special quirks

The real-world ingredients: timeseries files

These are the files that make backtesting possible - real, measured historical data for that specific year, not anything invented by the model:

File	What it really is
load.csv	How much electricity Germany actually used, every hour of that year
solar_profile.csv / wind_onshore_profile.csv / wind_offshore_profile.csv / run_of_river_profile.csv	How much sun/wind/river-flow was actually available each hour (as a fraction of full capacity, e.g. 0.65 = 65% of maximum possible output)
biomass_profile.csv / other_res_profile.csv	Similar real profiles for biogas plants and other minor renewable sources
co2_price.csv, hard_coal_price.csv, natural_gas_price.csv, oil_price.csv	What fuel and carbon-emission allowances actually cost that year
*_outage.csv / *_must_run.csv	How much of each power-plant fleet (coal, gas, nuclear, etc.) was actually out of action for maintenance/breakdowns, or forced to keep running at a minimum level, each hour

Important idea to hold onto:

None of these input files are produced by AMIRIS. They are real, historical, measured facts. AMIRIS only calculates ONE new thing from them: the electricity price. That is the entire point of backtesting - checking whether that one calculated thing matches reality.

5. What Comes Out: the Result Folder

After a simulation finishes, all its answers land in a results folder. The most important files there are:

File	Plain-language meaning
output.pb	The master answer file, in a compact computer-only format - everything else in this folder is extracted from it
metadata.json	An instruction manual describing what every column in every spreadsheet means
DayAheadMarketSingleZone.csv	THE key file: the electricity price AMIRIS calculated for every single hour of the simulated year. This is what we compare against real history.
ConventionalPlantOperator.csv	What the coal/gas/nuclear/oil power plants did each hour: how much they produced, how much money they made, how much CO2 they emitted
RenewableTrader.csv / NoSupportTrader.csv / SystemOperatorTrader.csv	What the various types of wind/solar/biomass sellers did each hour, including any government support payments received
GenericFlexibilityTrader.csv	What battery-storage operators did: when they charged, when they discharged, and at what price
DemandTrader.csv	How much electricity buyers requested and were given each hour

6. How Everything Links Together

Put together, the whole process flows in one direction, like an assembly line:

- Step 1 - Real history goes in: the timeseries files (real demand, real weather, real fuel prices) for a chosen year are placed in the scenario's recipe folder.
- Step 2 - The recipe describes the players: the agents and contracts files describe every virtual power plant, trader, and buyer, and the rules for how they interact.
- Step 3 - The engine runs the year: the Java engine steps through all 8,760 hours, having every agent bid and trade, and works out the market-clearing price each hour.
- Step 4 - Results are extracted: the raw output is converted into the readable CSV spreadsheets described above, landing in the result folder.
- Step 5 - We check reality: we pull out the calculated price column and place it side by side with the REAL historical price for that year (downloaded separately, since AMIRIS does not know or store what really happened - only we can check that).

7. The Thesis Plan: What We Set Out to Do

The overall research question: to what extent can AMIRIS reproduce real German electricity prices, and how does its accuracy compare with the commercially purchased data the department currently relies on? This breaks down into two separate comparisons:

Comparison 1: Backtesting (is AMIRIS trustworthy at all?)

Run AMIRIS for several past years where we already know the real price, and measure how close the simulated price gets. This is the foundation - if AMIRIS cannot reproduce known history, there is no point comparing it to anything else.

Comparison 2: Benchmarking (is AMIRIS as good as what's being paid for?)

Compare AMIRIS's simulated price against the Energy Brainpool data the department has already purchased, for whichever year(s) that data covers - plus the real price for that year, if it has since become the past. This produces a genuine three-way comparison: reality, the paid product, and the free alternative.

The rough plan, in time

- Early weeks: set up the toolchain, reproduce a baseline year, build reusable comparison tools, write the model-description chapter while everything is fresh.
- Middle weeks: obtain and verify the Brainpool data; parameterise and backtest whichever additional year is needed; write the methodology chapter.
- Later weeks: run the three-way benchmarking comparison; write the results, discussion, introduction and conclusion chapters.
- Final days: formatting, references, and a reserved buffer.

8. What Has Actually Been Done So Far

This section is a plain narrative of the practical work completed to date - including the mistakes made and fixed along the way, since those are as instructive as the successes.

Setting up and running the first simulation

The AMIRIS engine and its Python toolbox were confirmed installed and working. The Germany2018 backtest scenario (one of the ready-made recipes that ships with AMIRIS) was run successfully, producing a full simulated year of results.

Getting real prices to compare against - including two false starts

To check the simulation, we needed the REAL historical German electricity price for comparison. Two data sources were tried:

- ENTSO-E (a pan-European market data platform): the wrong market 'zone' was initially

selected (a technical mix-up, easily made since many similarly-named zones exist) - caught and corrected before any wrong conclusions were drawn.

- SMARD.de (Germany's official market data platform, run by the national energy regulator): the first download accidentally came back with the wrong dates entirely (the website silently defaulted to 'today' instead of the requested historical year) - also caught, by checking the actual dates inside the downloaded file before using it, and fixed by re-downloading with the date range properly applied.

Why mention the mistakes:

Both errors were caught by checking the data itself rather than assuming the download worked - a habit worth carrying into the rest of the thesis. Neither error affected any of the results reported below, since both were fixed before any analysis was done on the bad data.

A genuine historical complication: the 2018 market split

Germany and Austria shared one combined electricity price zone until 1 October 2018, when they were formally split into separate zones. This is a real historical fact, not a data error - it meant 2018's real price data had to be assembled from two different columns (one for January-September, one for October-December) and stitched together into one continuous year before it could be compared against the simulation.

Comparing simulated vs. real prices

A reusable comparison tool was built that lines up the simulated price and the real price hour by hour, and calculates how closely they match (see Section 9 for what the resulting numbers mean). This was first done for 2018, then repeated for all other years AMIRIS ships ready-made recipes for: 2015, 2016, 2017, and 2019 - giving five full years of backtesting evidence rather than just one.

Digging into why the fit isn't perfect

Rather than stopping at 'the numbers are close but not perfect', the two biggest specific mismatches were investigated in detail, using the simulation's own detailed hour-by-hour records. Both were tracked down to concrete, understandable causes - explained fully in Section 10, since this is genuinely one of the most valuable findings of the work so far.

9. The Results: How Close Did AMIRIS Get?

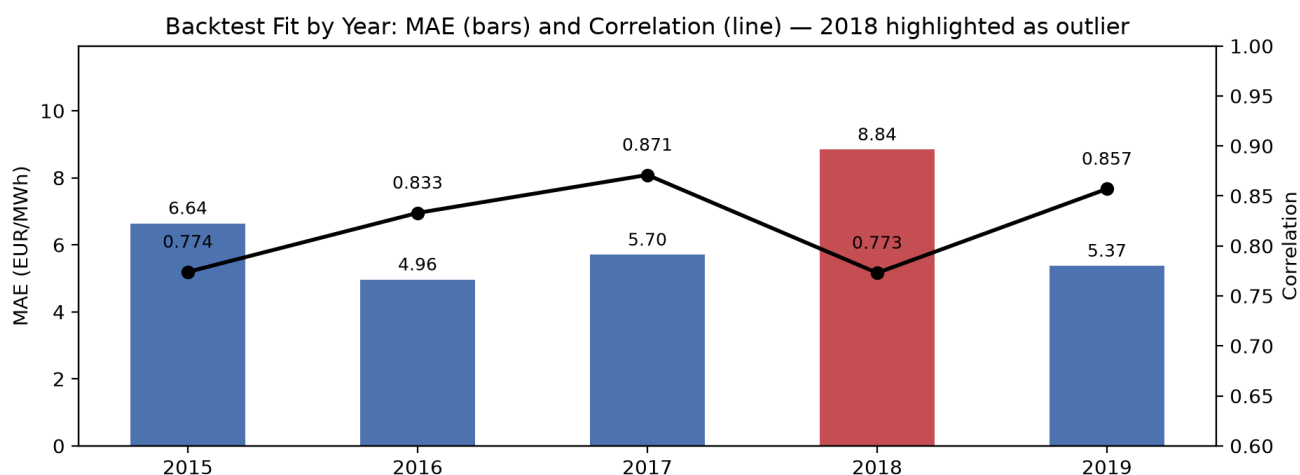
Three numbers are used throughout to measure 'closeness'. Before looking at the actual results, here is what each one means in plain terms:

Measure	What it tells you
MAE (Mean Absolute Error)	On an average hour, how many Euros per MWh the simulated price differs from the real price - ignoring whether it was too high or too low, just the size of the miss. Smaller = better.

Measure	What it tells you
RMSE (Root Mean Squared Error)	Similar to MAE, but large individual mistakes count extra heavily. If RMSE is noticeably bigger than MAE, it means most hours are close but a few hours are badly off.
Correlation	A score from 0 to 1 measuring whether the simulated price rises and falls in the same pattern as the real price over time, regardless of the exact amount. 1 = the timing of ups and downs matches perfectly; 0 = no relationship at all.

The five-year backtest results

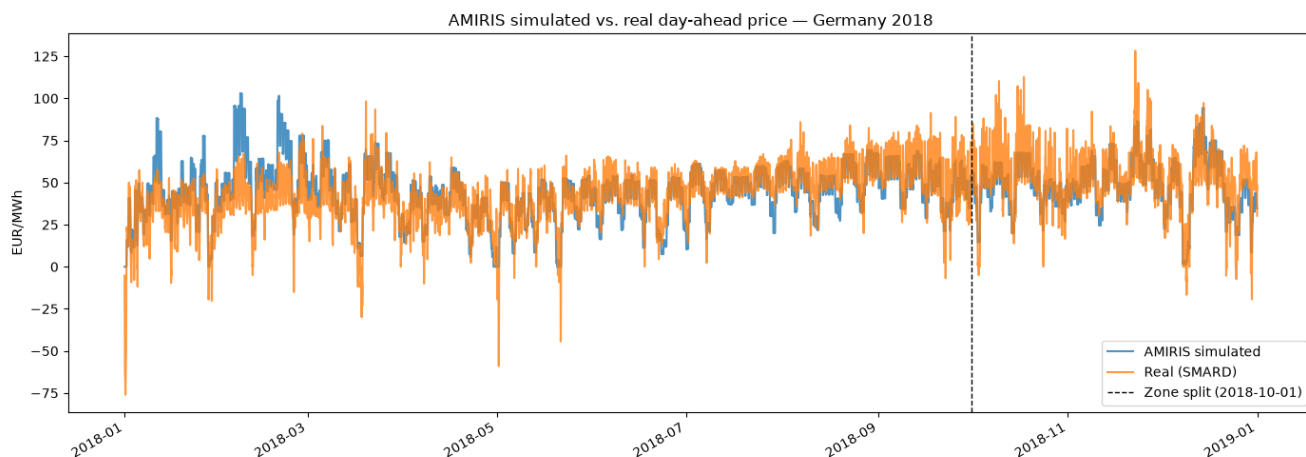
Year	MAE (EUR/MWh)	RMSE (EUR/MWh)	Correlation
2015	6.64	8.67	0.774
2016	4.96	6.92	0.833
2017	5.70	8.76	0.871
2018	8.84	11.72	0.773
2019	5.37	8.01	0.857



In plain terms: across all five years, the simulated price is typically within about 5-9 EUR/MWh of the real price on an average hour, and the year-to-year pattern (correlation of 0.77-0.87) shows AMIRIS reliably captures the general shape and timing of price movements, even where it doesn't nail the exact number. 2016 and 2017 fit best; 2018 fits worst - and, importantly, we now know two specific, concrete reasons why (Section 10), rather than just an unexplained gap.

What a good fit actually looks like

The chart below overlays the simulated price (blue) against the real price (orange) for every hour of 2018. Where the two lines sit closely on top of each other, the simulation is doing well; where they separate, that is a mismatch worth investigating.



10. What the Results Actually Mean: Two Confirmed Explanations

The 2018 result was investigated in detail because it was the weakest of the five years. Rather than leaving that as an unexplained number, the simulation's own detailed records were examined hour by hour, and two separate, genuine causes were identified and confirmed with evidence - not guesses.

Cause 1: The simulation won't let prices go negative on quiet holidays

On 1 January and 1 May 2018 (public holidays, when factories are closed and electricity demand is low), the REAL price fell dramatically below zero - down to about -76 EUR/MWh. This is a real and well-known phenomenon: when there is more wind and solar power available than anyone needs, and switching a wind turbine off is often more troublesome/costly than just giving the electricity away, sellers will actually pay buyers to take it, pushing the price negative.

The simulation, on those same hours, calculated a price of exactly 0.00 EUR/MWh - not negative. Looking at the detailed hour-by-hour records confirmed why: the model's 'virtual renewable-energy sellers' are currently only allowed to offer their electricity at a minimum price of zero, never below. So even though the simulation correctly recognised there was more clean power available than needed (some of it genuinely went unsold, exactly as in real life), it simply stopped at zero rather than following reality down into negative territory.

In short:

This is a specific, well-understood, and fixable setting inside the model - not a sign that AMIRIS is fundamentally unreliable. It only matters on a small number of unusual, very low-demand hours per year.

Cause 2: The simulation assumes Germany trades with nobody else

Between 5 and 19 February 2018, Germany experienced a cold spell, and real electricity demand rose by roughly 14% above its usual average. The simulation's price for these two weeks came out far too high (about 92-103 EUR/MWh, versus a real price of about 45-53 EUR/MWh).

Careful checking ruled out the obvious suspects: fuel prices during this period were completely normal, the carbon-emissions price was actually cheaper than usual, and power-plant availability was better than the yearly average - none of these can explain a price spike. The real explanation is structural: in reality, when German demand spikes, the country can import some extra electricity from neighbouring countries (France, Denmark, etc.) to help meet it cheaply. The version of AMIRIS used here has no concept of cross-border trade at all - it assumes Germany must generate 100% of its own electricity from its own power plants, no matter what. So when demand spiked, the simulation had no cheaper import option to fall back on, and had to rely on its own progressively more expensive backup power plants, pushing the calculated price higher than what really happened.

In short:

This is a genuine, structural limitation of the specific scenario setup being used (it models Germany in isolation), not a flaw in the model's core logic. It is also a well-defined, testable thing to potentially improve.

Why finding these two causes is a good outcome, not a bad one

A thesis that simply reports 'the numbers were close-ish' is far weaker than one that can say precisely when and why a model works well, and when and why it doesn't. Both causes found here are specific, evidenced, and point to concrete next experiments - which is exactly the kind of finding a research project is meant to produce.

11. A Smarter Version of the Comparison

After a meeting with the department, the plan for comparing AMIRIS against the purchased Energy Brainpool data was sharpened into something more rigorous than originally intended - worth explaining plainly, since it changes what happens next.

The problem with the original comparison plan

The backtesting work described in this guide so far (Sections 8-10) feeds AMIRIS real historical facts that AMIRIS itself goes and finds (from public sources like SMARD and ENTSO-E), then checks whether its calculated price matches what really happened. That is a fair test of AMIRIS overall, but it cannot cleanly separate two different possible explanations for any mismatch: was the mismatch caused by AMIRIS using slightly different input facts than Brainpool used, or by AMIRIS's internal decision-making being different from Brainpool's? Both would produce the same symptom - a price that does not quite match - so this design alone cannot tell you which one is actually responsible.

The improved plan: hold the ingredients fixed, compare only the cooking***Analogy:***

Imagine two chefs are given the exact same ingredients and asked to cook the same dish. If their

results still taste different, you know the difference comes from their technique, not from one of them secretly using a different ingredient. That is the logic of the new plan.

Rather than letting AMIRIS gather its own facts, the department will hand over the actual underlying assumptions Energy Brainpool itself used when it produced the price data the department purchased - things like how much solar and wind capacity was assumed to exist, what fuel prices were assumed, and so on. Those exact same assumptions will then be typed into AMIRIS, and AMIRIS will be asked to calculate a price from them. The result is then compared directly against Brainpool's own price - not against real-world history this time. If the ingredients are identical and the two prices still come out different, that difference can only be explained by the two models' internal logic being different - a much cleaner, more convincing test.

Why 2019 is the natural year to use

AMIRIS has already been backtested against real 2019 prices (Section 9 of this guide), and it was one of the best-fitting years. Reusing 2019 for this new comparison would complete a full three-way picture - AMIRIS vs. Brainpool vs. reality - all for the same year, using work that is already finished, rather than starting over on an unfamiliar year.

The immediate task: a shopping list for the department

Before any of this can happen, the department needs to know exactly what 'ingredients' AMIRIS requires, so they can check what they actually have in their Brainpool data and hand over the matching pieces. A dedicated one-page request document was produced for exactly this purpose, covering: how much electricity was needed each hour, details of every power plant type (how big, how efficient, how costly to run, how much they pollute, how often they break down), details of every renewable type (how big, and crucially, an hour-by-hour weather-driven output shape - not just a single capacity number), battery/storage details, fuel and carbon prices, and a few yes/no questions about how Brainpool's own model handles things like electricity trade with neighbouring countries.

An honest caveat included in that request:

It is quite likely Brainpool's data will include the big-picture numbers (like total solar capacity) but not the more detailed hour-by-hour weather shape AMIRIS also needs. Wherever something is missing from Brainpool's data, AMIRIS's own real-world data will be used to fill the gap instead - and exactly where that happens will be written down clearly, since it is a second honest explanation (alongside genuine differences in model logic) for any mismatch that shows up later.

12. What Happens Next

- Wait for the department to return the cross-referenced data request, showing which AMIRIS inputs their Brainpool holdings already cover.
- Do a gap analysis: for every required input, mark it as supplied, partly supplied, or needing to be filled in with AMIRIS's own real-world data instead.
- Build the 2019 scenario using Brainpool's own assumptions wherever available, run AMIRIS,

and compare its price against both Brainpool's price and the real 2019 price.

- If the two match closely, that is real evidence the department could use AMIRIS as a free substitute for Brainpool going forward, once a proper input-data pipeline exists. If they diverge, the same detective work already used to explain the 2018 mismatches (Section 10) will be applied to work out exactly why.
- Two smaller technical experiments remain useful regardless of the above: allowing the simulation's renewable sellers to bid negative prices on quiet holidays, and adding a simple adjustment for cross-border electricity trade.
- Writing continues in parallel with the technical work, following the thesis's own rule of spending roughly 40% of total time on writing throughout, rather than only at the end.